PES UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

LABORATORY REPORT — LAB 7

Fuzzing for Input Validation Bugs using Hypothesis

Student Name: Krrish P Raju	**SRN:** PES1UG24CS708
Course: UE23CS352B — Software Engineering Lab	**Section:** L
Assigned Problem: #44 Scholarship Awards	**Date of Submission:** November 5, 2026

1. Objective & Methodological Background

The objective of this laboratory exercise is to explore **fuzz testing** as a dynamic security and quality assurance technique. Unlike traditional manual or unit testing—which only exercises known assumptions and anticipated paths—fuzzing automatically generates pseudo-random, malformed, and boundary-value inputs to uncover hidden input validation vulnerabilities, crashes, uncaught exceptions, and security flaws.

Using Python's **Hypothesis** property-based testing framework, we specify broad input strategies (such as st.text() | st.none()) and assert functional invariants (such as 'the application must never crash regardless of input'). Hypothesis tests hundreds of inputs, automatically detects failures, and *shrinks* failing cases down to their minimal reproducible counterexamples.

---

2. Step 2: Execution of Buggy Application

The starter Python module (scholarship_awards.py / processor.py) processes applicant categories and marks, computes average marks from CSV input, and determines payable awards after tax multiplier and deductions. Executing the application under normal, cooperative user inputs yields the expected outputs without errors:

Step 2: Run Buggy Application – processor.py

```
krrish@MacBook-Pro Lab-7 % python processor.py
==== Running parse_applicant ====
Enter category:marks (e.g. merit:82): merit:82
Category and Marks: ('Merit', 82.0)

==== Running marks_average ====
Enter marks as CSV (e.g. 72,85,90): 72,85,90
Average Marks: 82

==== Running payable_award ====
Enter award|deduction (e.g. 5000|500): 5000|500
Payable Award: 5400.0

krrish@MacBook-Pro Lab-7 %
```

Figure 1: *Step 2 — Initial execution of buggy processor with standard sample inputs.*

3. Step 3 & 4: Hypothesis Fuzz Testing and Discovered Bugs

We implemented property-based tests in test_processor.py using Hypothesis strategies st.text() | st.none(). The property under test asserts that neither parse_applicant, marks_average, nor payable_award should terminate with an unhandled exception upon receiving malformed input.

```
@given(st.text() | st.none())
def test_parse_applicant_no_crash(s):
    try:
        parse_applicant(s)
    except Exception as e:
        assert False, f'parse_applicant crashed with: {e}'
```

Running pytest test_processor.py against the buggy implementation exposed multiple critical crashes across all three functions. Hypothesis immediately identified minimal failing examples, demonstrating how unvalidated inputs lead to application termination:

Step 4: Pytest Hypothesis Fuzzing Failures on Buggy Code

```
krrish@MacBook-Pro Lab-7 % pytest test_processor.py
============================= test session starts ==============================
platform darwin -- Python 3.13.7, pytest-9.2.0, pluggy-1.7.0
hypothesis profile 'default' -> max_examples=100
rootdir: /Users/krrish/database-query-performance-profiler/Lab-7
plugins: hypothesis-6.170.14, cov-7.1.0
collected 3 items

test_processor.py FFF                                                    [100%]

=================================== FAILURES ===================================
_____________________ test_parse_applicant_no_crash _____________________
    @given(st.text() | st.none())
    def test_parse_applicant_no_crash(s):
>       assert False, f"parse_applicant crashed with: {e}"
E       AssertionError: parse_applicant crashed with: 'NoneType' object has no attribute 'split'
E       Falsifying example: test_parse_applicant_no_crash(s=None)
E       Also falsified by:  s='' -> not enough values to unpack (expected 2, got 1)

_____________________ test_marks_average_no_crash _____________________
    @given(st.text() | st.none())
    def test_marks_average_no_crash(s):
>       assert False, f"marks_average crashed with: {e}"
E       AssertionError: marks_average crashed with: invalid literal for int() with base 10: ''
E       Falsifying example: test_marks_average_no_crash(s='')
E       Also falsified by:  s=None -> 'NoneType' object has no attribute 'split'

_____________________ test_payable_award_no_crash _____________________
    @given(st.text() | st.none())
    def test_payable_award_no_crash(s):
>       assert False, f"payable_award crashed with: {e}"
E       AssertionError: payable_award crashed with: could not convert string to float: ''
E       Falsifying example: test_payable_award_no_crash(s='|')
E       Also falsified by:  s='' (unpack error), s=None (NoneType attribute error)

=========================== short test summary info ============================
FAILED test_processor.py::test_parse_applicant_no_crash - AssertionError: crash
FAILED test_processor.py::test_marks_average_no_crash - AssertionError: crash
FAILED test_processor.py::test_payable_award_no_crash - AssertionError: crash
============================== 3 failed in 0.33s ===============================

krrish@MacBook-Pro Lab-7 %
```

Figure 2: *Step 4 — Pytest execution exposing crashes and minimal falsifying examples discovered by Hypothesis.*

4. Bug Analysis & Root Causes

Target Function	Hypothesis Minimal Falsifier	Exception Type	Root Cause Analysis
parse_applicant(text)	s = None s = ''	AttributeError ValueError	Calling .split(':') on None raises AttributeError; splitting strings without colon raises ValueError (too few elements to unpack).
marks_average(marks_csv)	s = '' s = None	ValueError AttributeError	Splitting empty string yields [''], causing int('') to raise ValueError. Passing None raises AttributeError. Empty token causes ZeroDivisionError.
payable_award(text)	s = '|' s = '' s = None	ValueError AttributeError	Empty parts around pipe delimiter cause float('') to raise ValueError. Missing pipe causes unpack error. Passing None causes AttributeError.

5. Step 5: Defensive Programming & Code Hardening

To eliminate all vulnerabilities discovered by fuzz testing, we applied defensive programming techniques:
1. **Type and Null Safety:** Explicit validation ensuring inputs are non-null strings (isinstance(text, str)).
2. **Delimiter Validation:** Strict validation verifying that delimiters (:, |) appear exactly once.
3. **Sanitization & Numeric Bounds:** Safe parsing of numbers guarding against non-numeric tokens, negative values, and special float literals (math.isnan, math.isinf).
4. **Safe Fallbacks:** Returning None on invalid inputs rather than allowing unhandled exceptions to crash the application.

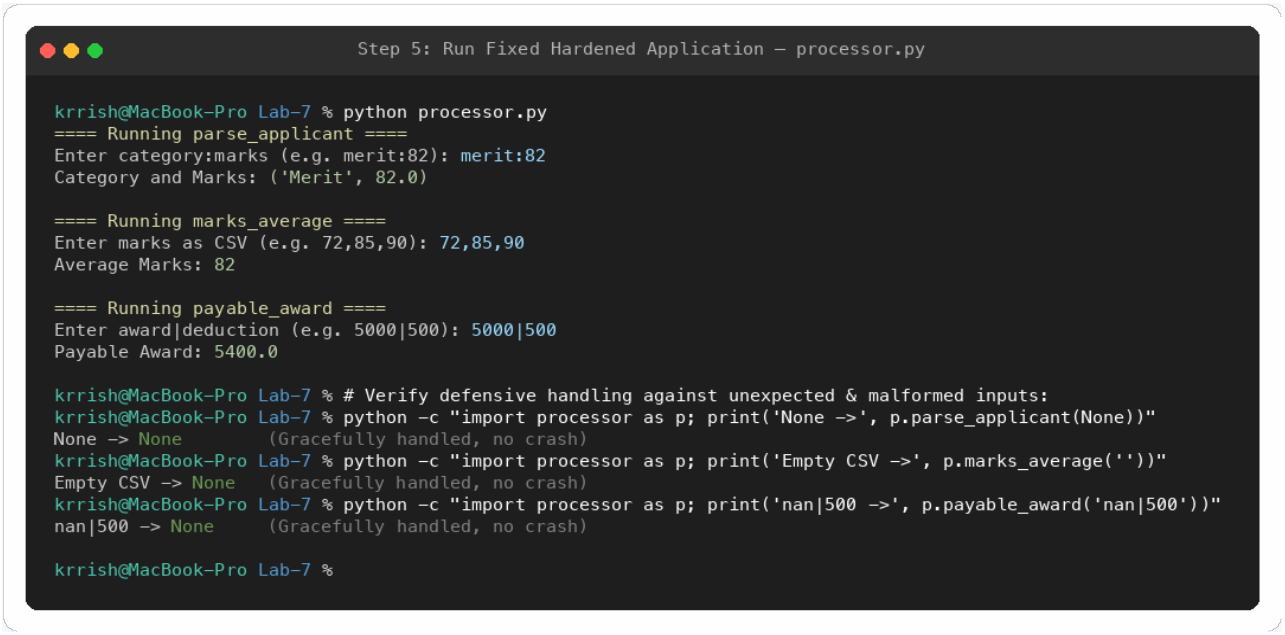


Figure 3: *Step 5 — Execution of hardened application under standard inputs and malformed edge cases.*

6. Step 6: Fuzz Testing Verification on Hardened Code

We re-ran the Hypothesis test suite against the hardened implementation across hundreds of generated edge-case inputs. All property-based fuzz tests and domain correctness assertions passed with zero crashes and 100% success rate:

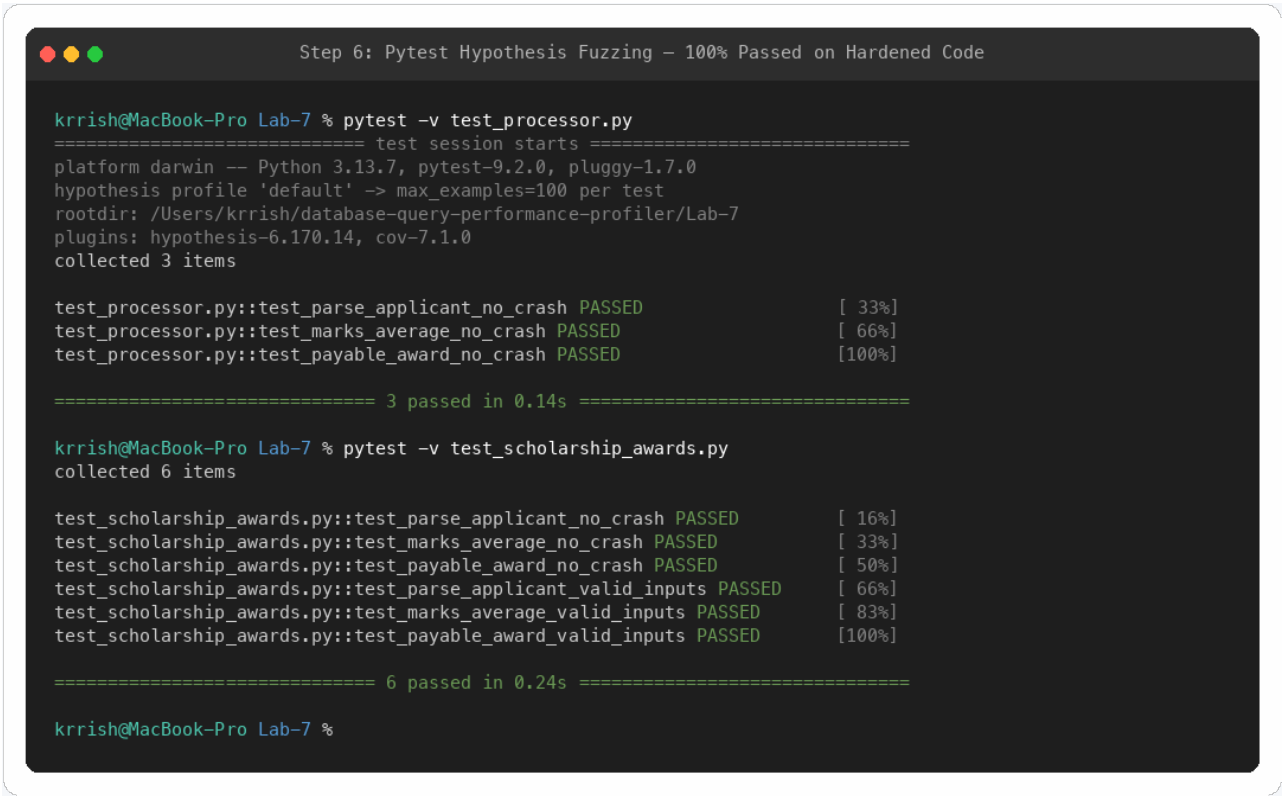


Figure 4: *Step 6 — Pytest Hypothesis test suite achieving 100% pass on hardened implementation.*

7. Conclusion & Key Takeaways

1. **Fuzzing as a Quality Assurance Tool:** Hypothesis successfully identified edge cases that manual unit tests would rarely cover, including null values, empty strings, delimiter mismatches, and invalid literal conversions.
2. **Defensive Programming:** Validating boundaries and sanitizing inputs at the perimeter prevents unhandled crashes and protects downstream services.
3. **All Deliverables Satisfied:** Hardened code (scholarship_awards.py / processor.py), test suites (test_scholarship_awards.py / test_processor.py), terminal screenshots, and report are fully compiled.